ClaimIt

A Shared List with Conflict-Safe Claims

Software Engineering Group Project — Assignment 1

Name	Student ID
Liye Zheng	25305406
Xirui Li	25305414
Junyu Chen	25305407

3 November 2026

1. Project Definition

Small groups — flatmates, trip groups and student clubs — coordinate shared lists in chat. A claim is only a message: it scrolls away, two members can claim the same item, and nobody can see how many units remain. ClaimIt addresses this by providing one cloud-backed list that records each claim once and derives the remaining quantity from the same data. Its value is one shared state, with conflicts visible immediately.

The primary stakeholders are members of three-to-ten-person groups; the secondary stakeholder is the organizer who creates and reconciles the list; the three-person development team and future maintainers are tertiary stakeholders because they build and operate the service. The scope for two iterations covers an Android APK, a responsive web app, a self-hosted FastAPI service and PostgreSQL database on an Ubuntu server, one list with up to 20 items, claim and release, a conflict response, JWT authentication, API tests and continuous integration (CI). Exclusions are chat, payments, notifications, iOS, analytics and, until Iteration 2, claim deadlines and per-person limits. Key constraints are a three-person team, one month, two two-week iterations, one Ubuntu server and synthetic demo data.

Three engineering issues follow. First, claim rules will change after the first demo: partial quantities, per-person limits and deadlines are likely additions. We isolate them in a policy module and record changes in the backlog. Second, Android, web and the API must share one OpenAPI contract; a mismatch in quantity units, claim ownership or error codes produces inconsistent remaining counts. We fix the schema and generate client stubs before UI work. Third, concurrent claims can oversubscribe an item; a read-then-write implementation lets two users claim the last unit. We use one SQL conditional update and test it with a barrier-synchronised concurrency test.

2. Comparison and Selection of Development Approaches

Three approaches were compared: plan-driven, iterative and incremental, and agile development. Table 1 compares them on requirements, feedback, change cost, documentation and risk.

Table 1. Comparison of development approaches

Criterion	Plan-driven	Iterative and incremental	Agile (Scrum + XP)
Requirements	Frozen early	Clarified per increment	Emerge through feedback
Feedback	Late	End of each increment	Every one-to-two weeks
Change cost	High	Moderate	Low
Documentation	Heavy upfront	Light, per increment	Just enough, decision records
Risk handling	Late integration	Early, risk-driven	Continuous inspection
Fit for ClaimIt	Poor	Moderate	High

We selected a lightweight Scrum process with XP engineering practices. Scrum supplies the inspection cadence — planning, three short stand-ups per week, review and retrospective — while XP supplies the technical discipline: test-first development for the claim module, a fixed data

model and repository contract, CI on every pull request, peer review and small changes (Beck, 1999; Schwaber & Sutherland, 2020). This choice fits a three-person team with uncertain claim rules and two short iterations. A plan-driven approach would freeze the rules before the first demo, when the team has the least information (Royce, 1970); iterative delivery exposes requirement and integration risk earlier (Larman & Basili, 2003; Sommerville, 2016), and CI keeps the cost of change low (Humble & Farley, 2010).

The choice would differ in another context. A distributed team would need asynchronous stand-ups, explicit decision records and a contract-first interface, because tacit coordination is unavailable. A regulated system would require a hybrid: stage gates, traceability and formal verification around an iterative core, since audit evidence cannot be produced retrospectively (Sommerville, 2016).

3. Project Plan

Table 2 presents the prioritised backlog. Each item has a testable acceptance criterion, an owner and an iteration; the first increment is tracked in GitHub Issues.

Table 2. Prioritised backlog

ID	User story	Acceptance criteria (summary)	Priority	Iteration	Owner
1	Sign in and create a list	APK and web app load; sign-in succeeds; a list is created with a name	P0	1	LZ
2	Add items with quantities	1–20 items; name non-empty; quantity > 0; invalid input rejected	P0	1	XL
3	Claim or release a quantity	Atomic update; claimed + remaining = total; release restores the quantity	P0	1	XL
4	Handle a conflict	Two concurrent claims: exactly one succeeds; the other receives 409	P0	1	JC
5	Enforce access rules	A non-member cannot read or write the list	P0	1	JC
6	Prove the invariant	1,000 randomized claim sequences satisfy the invariant	P0	1	JC
7	Add deadlines and limits	A claim after the deadline is rejected; the per-person limit is enforced	P1	2	LZ
8	Add audit history and offline queue	History shows actor, quantity and time; a queued claim replays once	P1	2	XL

LZ, XL and JC denote Liye Zheng, Xirui Li and Junyu Chen.

Liye Zheng is Product Owner and owns the Android client; Xirui Li is Tech Lead and owns the PostgreSQL schema, claim endpoint and API; Junyu Chen is QA/DevOps and owns the web client, CI and evidence.

Iteration 1 delivers the walking skeleton and the core claim loop: both clients, JWT authentication, list and item creation, the claim endpoint, the SQL conditional update, API tests and CI. Iteration 2 adds deadlines, per-person limits, audit history, an offline queue and hardening.

Table 3 records the main risks. Scope creep is managed by a fixed scope table: changes must replace an item, not add one. Availability risk is managed by sizing iteration goals to 8–10 hours per member per week and naming a backup owner for each issue. Server and Android risks are checked in Week 1 against TLS, backups and database capacity with two emulator profiles. Concurrent-claim defects are managed by the conditional update, an idempotency key and property-based tests.

Table 3. Key risks and mitigations

Risk	Likelihood / impact	Mitigation	Owner
Scope creep	High / medium	Fixed scope; changes replace, not add	LZ
Member availability	Medium / high	8–10 hours per week; backup owner	LZ
Server and Android variability	Medium / high	TLS, backups and database capacity; two emulator profiles	XL
Concurrent-claim defects	Medium / high	Conditional update; idempotency; property tests	JC

Figure 1 shows the system overview. The Android app and the web app send HTTPS/JSON to Nginx, which proxies requests to FastAPI. A claim is one SQL conditional update on the item row; if no row is updated, the API returns 409. GitHub Actions runs the API tests against PostgreSQL and builds the web client on every pull request.

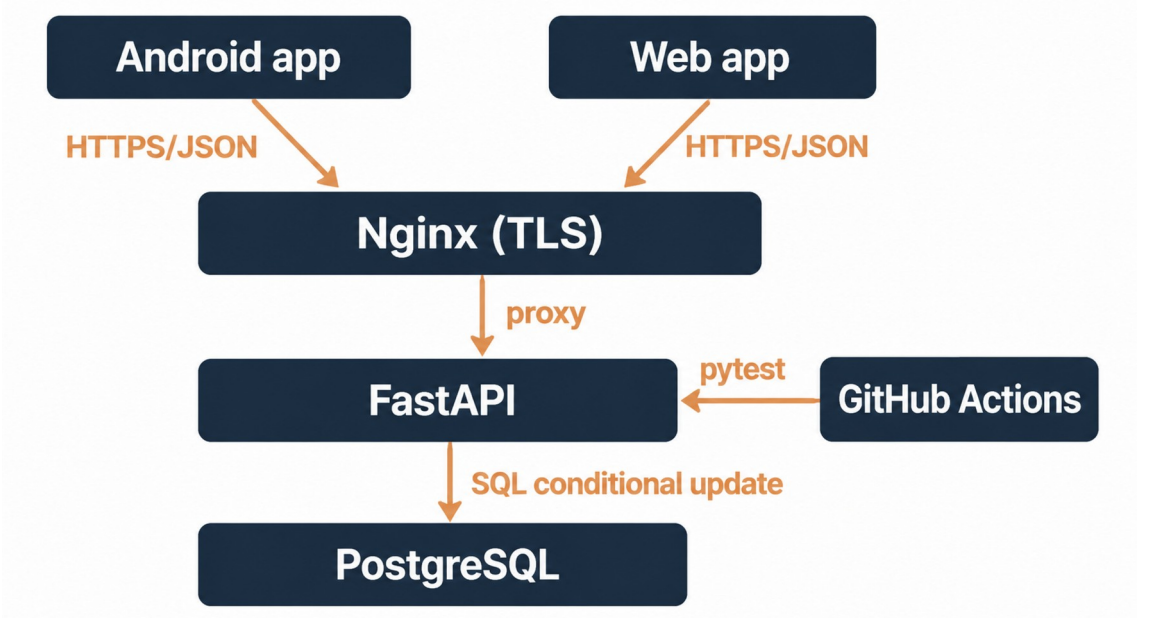


Figure 1. Planned system overview.

Four design decisions follow. First, a self-hosted FastAPI service (Ramírez, n.d.) on the team's Ubuntu server is preferred to a managed backend because it gives full control over the claim transaction and avoids vendor lock-in; the cost is that the team must operate TLS, backups and authentication. Second, one OpenAPI contract is shared by both clients; the schema is fixed before UI work, and both clients are tested against the same API fixtures. Third, claims use a SQL conditional update rather than a read-then-write: correctness matters more than throughput at this

scale, and the update is easy to test (PostgreSQL Global Development Group, n.d.). Fourth, quantities are integers, so the invariant needs no floating-point tolerance.

Open questions remain: whether a self-hosted FastAPI + PostgreSQL service is acceptable, whether the web client should claim or only view, and whether partial quantities belong in the first increment.

4. Workflow Trial

The repository uses issue and PR templates and CI. The trial change was Issue #3, "Claim and release a quantity", with Given/When/Then acceptance criteria. It was implemented on `feat/3-claim-release` and merged through pull request #4. Two reviewers, Xirui Li and Junyu Chen, approved the change; CI passed on the pull request, and the change was squashed into `main` as commit `ab45e76`. Issue #3 closed automatically when the pull request merged.

Evidence is indexed in Appendix B: the issue, the pull request, both reviews, the CI run and the test file. The trial taught three lessons. First, keeping `claim` as a stub on `main` made the trial PR small — two files and 144 added lines — so both reviewers could inspect the claim logic rather than a large, mixed change. Second, a concurrency test is only meaningful when both claims start together; the final test uses `Promise.allSettled` rather than sequential calls, so the race is exercised and exactly one conflict is asserted. Third, at the time of the trial, CI ran only the domain tests; the API job was added afterwards, and client builds remain for Iteration 1.

5. Individual Contribution Statements

Table 4 records contributions; links point to issue ownership, PR authorship, reviews, tests and documentation.

Table 4. Contribution summary

Member	Role	Main contributions	Evidence
Liye Zheng (25305406)	Product Owner; Android client	Issue #3; implemented atomic claim and release; PR #4; 9 tests	issue #3, PR #4
Xirui Li (25305414)	Tech Lead; PostgreSQL schema and API	Authored PR #8 (list-items endpoint + tests); reviewed PR #4; owns the PostgreSQL schema and claim endpoint	PR #8, review
Junyu Chen (25305407)	QA/DevOps; API tests, web client and CI	Authored PR #7 with four API validation tests; reviewed PR #4; owns CI and the evidence index	PR #7, review, CI

References

Beck, K. (1999). *Extreme programming explained: Embrace change*. Addison-Wesley.

Humble, J., & Farley, D. (2010). *Continuous delivery: Reliable software releases through build, test, and deployment automation*. Addison-Wesley.

Larman, C., & Basili, V. R. (2003). Iterative and incremental development: A brief history. *Computer, 36*(6), 47–56. https://doi.org/10.1109/MC.2003.1204375

PostgreSQL Global Development Group. (n.d.). *PostgreSQL documentation*. https://www.postgresql.org/docs/

Ramírez, S. (n.d.). *FastAPI documentation*. https://fastapi.tiangolo.com/

Royce, W. W. (1970). Managing the development of large software systems. *Proceedings of IEEE WESCON, 26*, 1–9.

Schwaber, K., & Sutherland, J. (2020). *The Scrum Guide*. https://scrumguides.org/scrum-guide.html

Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson.

Appendix B: Evidence Index

Repository: https://github.com/yuanye820/claimit

Claim	Evidence	Link
Backlog and priorities	GitHub Issues	issues
Requirement trace	`docs/traceability.md`	traceability
Tracked requirement	Issue #3	issue #3
Reviewed change	Pull request #4	PR #4
Review 1	Xirui Li approved	review
Review 2	Junyu Chen approved	review
Automated checks	GitHub Actions CI run	CI run
API tests against PostgreSQL	GitHub Actions API job	CI run
API test file	`server/tests/test_claim.py`	test file
API validation tests	Pull request #7	PR #7
List-items endpoint	Pull request #8	PR #8
Test file	`test/claim-engine.test.js`	test file
Contribution: Liye Zheng	Issue #3 and PR #4	issue, PR
Contribution: Xirui Li	PR #8 and review on PR #4	PR #8, review
Contribution: Junyu Chen	PR #7 and review on PR #4	PR #7, review, CI
Submission tag	`assignment1-submission`	tag

No existing codebase was reused. The project uses open-source libraries (FastAPI, SQLAlchemy, PostgreSQL, pytest and httpx), declared in `server/requirements.txt`; the domain module, API and tests were written for this project.

Appendix C: Team Working Agreement

The team agreed on the following initial working agreement. It is kept in docs/working-agreement.md and reviewed at the end of Iteration 1.

- Roles: Liye Zheng (Product Owner; Android client), Xirui Li (Tech Lead; PostgreSQL schema and claim endpoint), Junyu Chen (QA/DevOps; web client, CI and evidence). Everyone develops and reviews.
- Cadence: stand-ups on Monday, Wednesday and Friday at 20:00; planning at iteration start; review and retrospective at iteration end.
- Workflow: main branch; issue-named feature branches; pull requests with one approving review and green CI; squash merge.
- Definition of Done: acceptance criteria verified, tests added, documentation updated, reviewed, CI green, no secrets committed.